Computer Graphics: Lecture 17

Spheres, Cube-maps, and Skyboxes

Welcome back!

Last time we learned the whole Phong model

We learned how to draw not only directional lights, but point lights as well. [what's the difference?]

We also learned about storing structs in WGSL, how it can help organize the software design of our shaders but how it also requires understanding *alignment*. [how does alignment work?]

This time

We're going to learn about generating spheres.

Really? That's all?

That's all…but it's a bit more challenging than you think.

We're also going to learn a new texturing technique: the cube map. Finally: no more duplicating the UV coordinates on a cube.

Cube mapping isn't just useful for texture mapping cubes: it's great for spheres, too, and it even enables some cool features such as skyboxes and environment maps (shiny textures that reflect the environment).

Spheres

Okay, this is a geometry pop quiz. If you had to generate a sphere mesh *right now*, what would you do?

[Let's think about this as a class.]

Circles

Chances are, learning to generate a circle will help us understand how to generate a sphere.

So let's start with generating a circle.

Ultimately, our video card wants to draw triangles. It is technically possible to use fancy tesselation shaders to generate a circle that is visually perfect (i.e., at a given resolution it is impossible to tell it from an ideal circle).

But let's not worry about that now. Let's just generate a regular polygon with a lot of sides that we will interpret as a circle if it has enough sides.

Circles (2)

The most straightforward way to generate a circle is to first decide how many sides we want. Say, 100. Then, plot that many points in the right place.

```
const verts = []; // vertex data: x,y coordinates
const nPoints = 100;

for (let point = 0; point < nPoints; point++) {
    ...
}
```

So…[how do we know where a point goes? What are its coordinates?]

Circles (3)

These points are going to go around the circle. Over the course of this process, the points will complete a single turn.

That means, the first point will start at 0 turns, the second will then be at one hundredth of a turn, the third will be two hundredths, etc.

So, we can determine the number of turns like this:

```
for (let point = 0; point < nPoints; point++) {
    const turns = point / nPoints;
}
```

But that's not enough. How do we turn *turns* into XY coordinates?

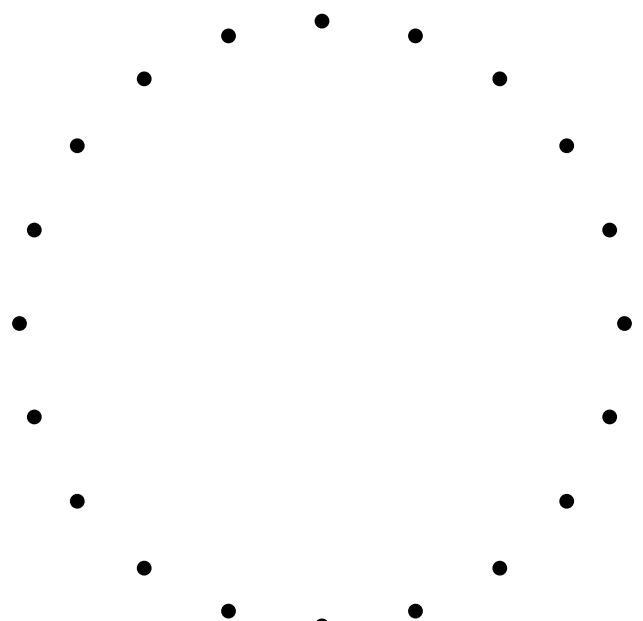
Circles (4)

First we need to convert them into angles in radians. Then we can use sine and cosine on the radians:

```
for (let point = 0; point < nPoints; point++) {
    const turns = point / nPoints;
    const rads = turns * TAU;
    const x = Math.cos(rads);
    const y = Math.sin(rads);
    verts.push(x, y);
}
```

Circles (5)

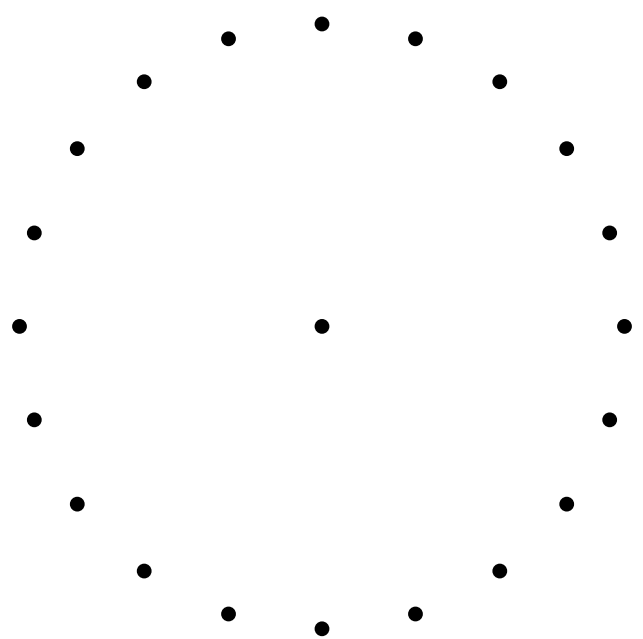
I coded the software renderer in my slides software to follow this algorithm using 20 vertices. It's pretty close to a circle.



Circles (6)

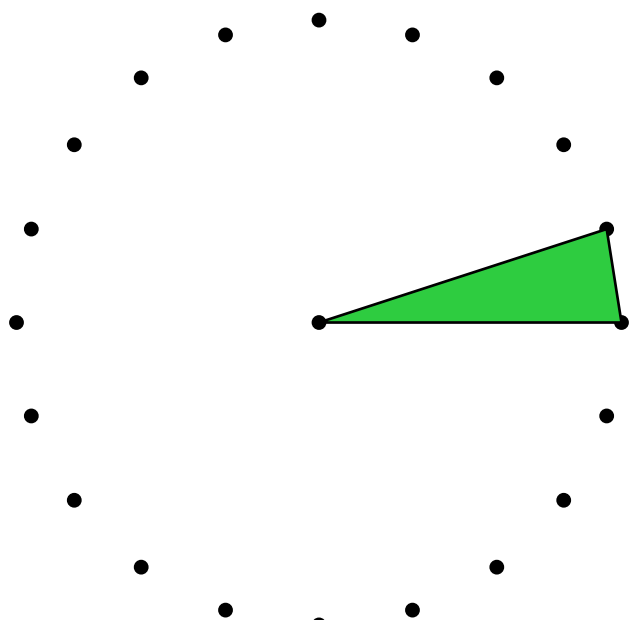
But a mesh isn't just a collection of points. It's also a collection of faces! (i.e., triangles). So how can we stitch our mesh together into triangles?

You might think we need to put a vertex in the center like this:



Circles (7)

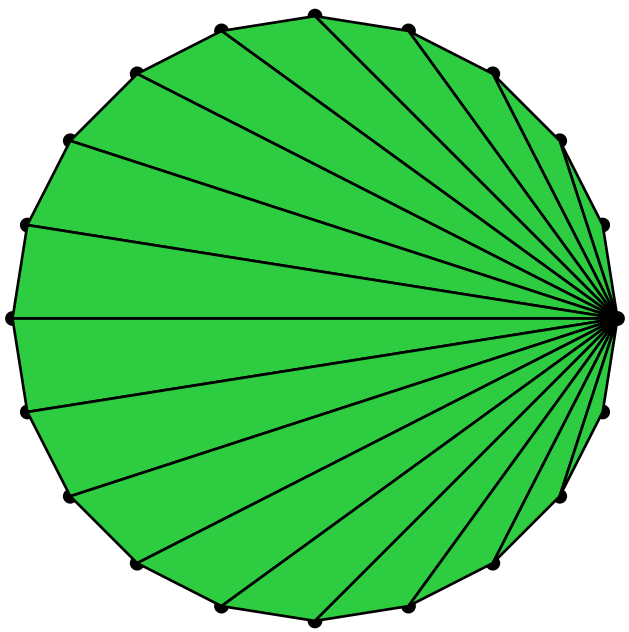
Then, we can stitch together a circle by creating triangles that pass through the center…



I.e., do this for all 20 slices.

Circles (8)

However, there is another way, where we just use one of the points on the side as the start of the triangle.



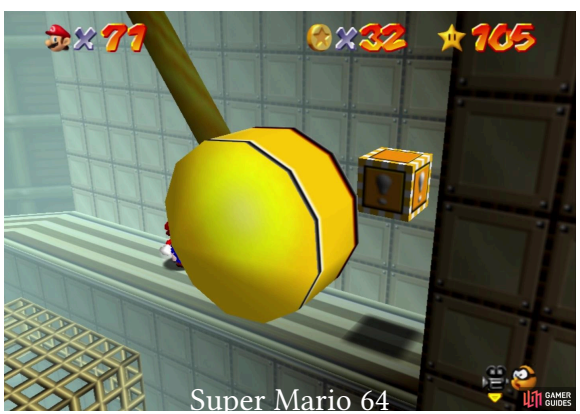
This saves a whole vertex 😮. Despite looking weird, it's just as good.

Triangle fans

There's a name for this kind of topology: a triangle fan.

It was built into legacy graphics APIs like OpenGL.

The first index in the strip would be the shared vertex, and then the other indices would be specified in the winding order. [0, 1, 2, 3, ...]



The pendulum's disc in Tick tock clock was a triangle fan.

Triangle fans (2)

Triangle fans were useful because you could generate any convex polygon from them. As long as the polygon was convex, you could use any point as the “center” point.

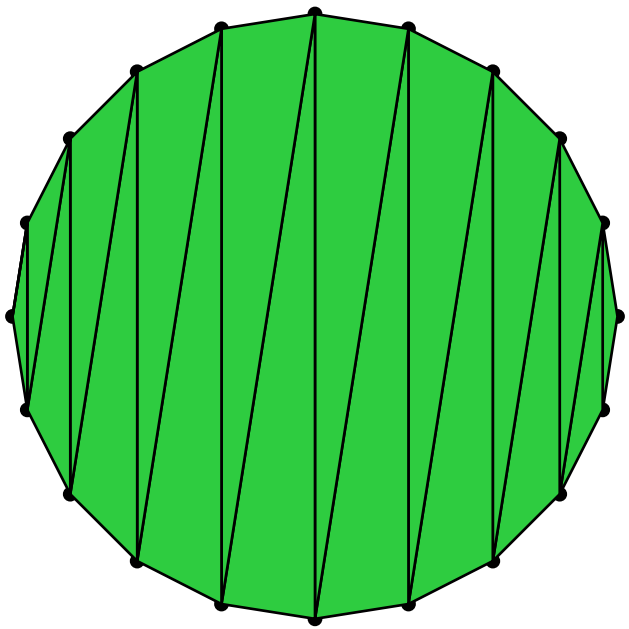
The screenshot before used the west most point in the disc.

However, WebGPU does not support triangle fans.

The reason is that it’s actually possible to use a triangle strip anywhere you would use a triangle fan, with the same number of indices.

[can anyone think of how to do it?]

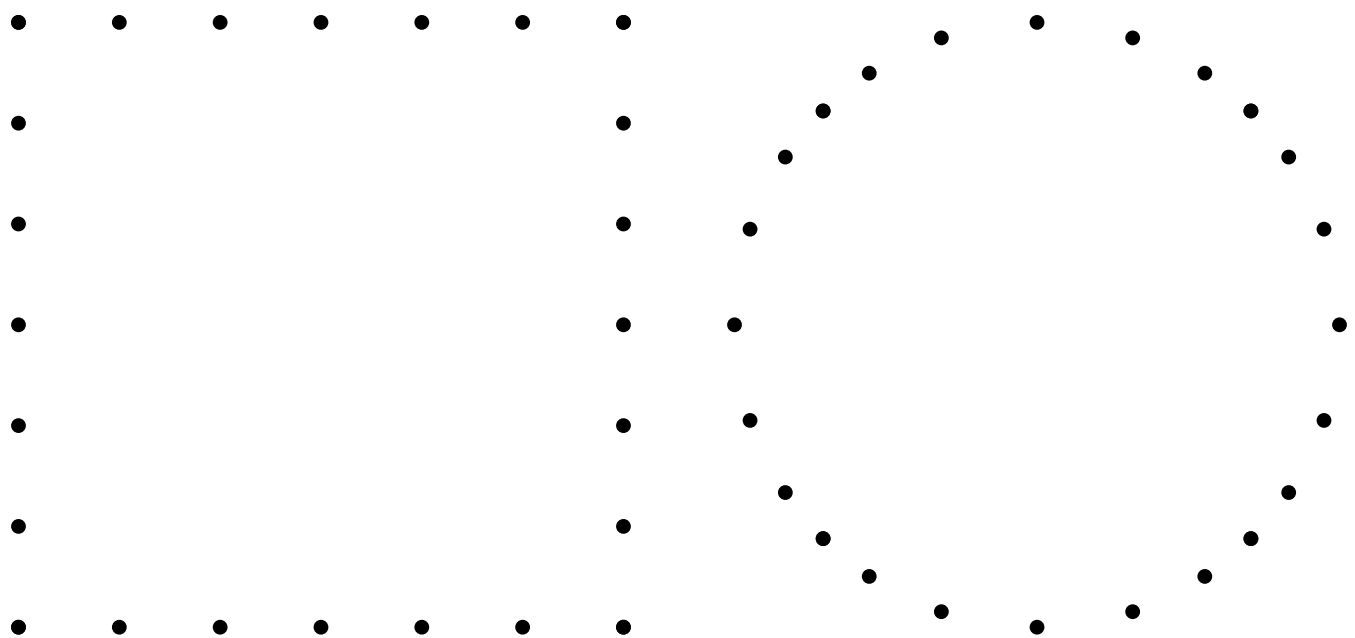
Triangle fans (3)



This saves a whole vertex and requires exactly as many indices as vertices! Just as good as a fan. [0, 1, 19, 2, 18, …]

Circles: more ways

There are actually several ways to generate circles. We can even start with a square and smoosh the points into a circle by normalizing them:



Before After

Circles: more ways

That's not ideal. It ends up with the point distribution being uneven, as you can maybe see.

However, it turns out that this is a great way of creating a sphere. This is called a cube sphere, and it's the *second* sphere algorithm we're going to learn today. Keep it in mind!

But first…

Questions?

Now spheres

Let's try to extend our algorithm for making circles.

Imagine that we made the equator of a large sphere that way.

Could we imagine how to stack more rings around the equator?

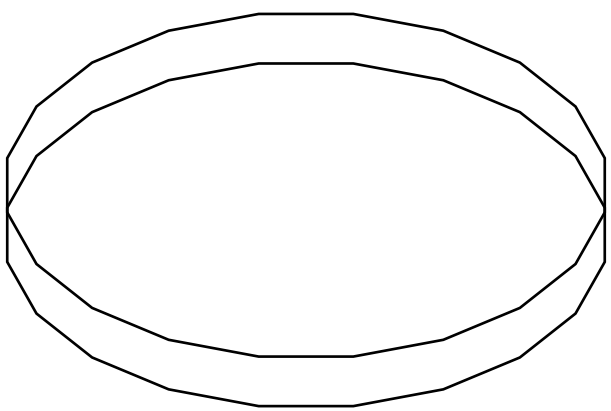
This kind of sphere is called a **UV-sphere**, because it's easy to compute the UV coordinates of its vertices.

[But how do we build one?]

UV-spheres

UV spheres are built out of rings, like the ones we generated earlier.

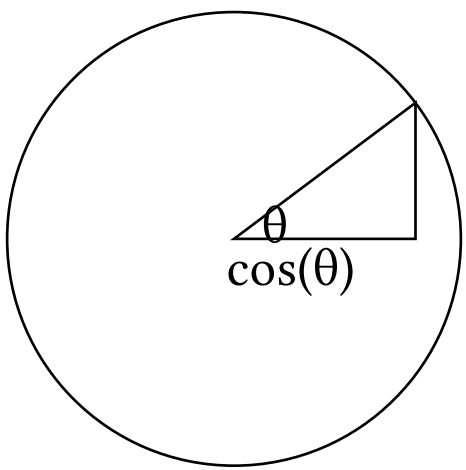
Only, we don't fill these rings into circles. We'll see how to tesselate them later.



Here are two rings. Assume the bottom is the equator. How do we shrink the top one to make it start to curve like the top of a sphere?

UV-spheres (2)

Let's imagine a vertical cross section. What we want is the cosine of the angle between two layers in the sphere:

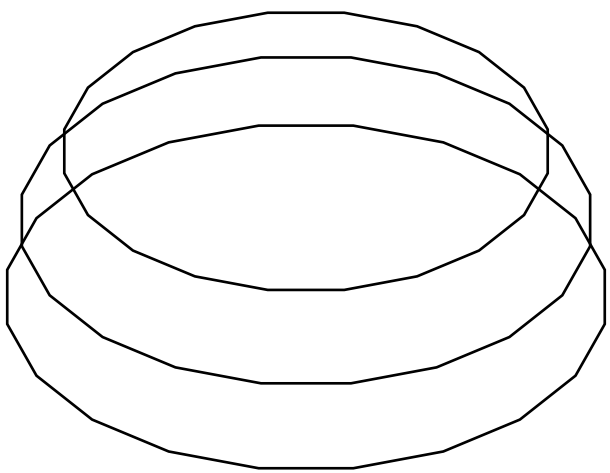


We can choose the angle of the height of the ring, or generate it. Either way the cosine of that angle is the radius of the ring.

UV-spheres (3)

And the y value is the sine of the angle.

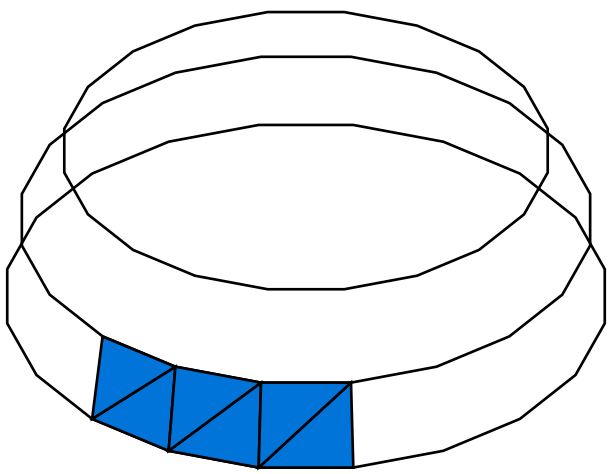
Let's stack 3 rings for good measure:



Hopefully we see the upper hemisphere start to form. But how do we form the triangles?

UV-spheres (4)

We pair adjacent rings together into strips. Then, we can draw a triangle strip all around the sphere. Here are some triangles: this pattern repeats:



We can draw the strip several ways. For example, if the points go clockwise: [0, pointsPerRing, 1, pointsPerRing + 1, 2, ...]

UV-spheres (5)

But what happens when we get to the top?